Project 2: Modelling daily cycling demand in Edinburgh

Group members: [Karthik Sutheeshkumar], [Kiet Lam], [Leon]

Word count: [0 words]

1 Introduction

Edinburgh operates a network of automatic cycling counters at strategic locations across the city that records the total number of cyclists passing each site daily, The resulting dataset contains 2,180 observations spanning from 2020 to 2025, covering a period that includes the COVID-19 pandemic and a gradual return to normal travel patterns. Using this data, we evaluate a sequence of linear regression models of increasing complexity rigorously to explain how daily cycling demand varies with weather conditions, day of week, seasonality, and longer-term trend, evaluating each model using leave-one-year-out cross validation. The analysis is guided by three questions of practical interest to Edinburgh City Council: how well the model predicts demand across different seasons, how sensitive cycling is to temperature, and what the long-term trend implies for infrastructure planning over the next decade.

2 Exploratory data analysis

Three derived variables are created before modelling. First, month is converted to an ordered factor with levels 1–12, ensuring plots display January to December in the correct order. Second, day of week is re-specified with explicit levels from Sunday to Saturday, overriding R's default alphabetical ordering. Third, a numeric trend variable is derived as the integer number of days elapsed since 1 January 2020, so that trend = 0 corresponds to the start of the observation period and increases by 1 for each subsequent day. This allows the model to capture any linear drift in cycling demand over 2020–2025.

Summary statistics table

The above table summarises the key numeric variables across the 2180 daily observations. Daily cyclist count average around 10,816 but with a large standard deviation of 4729 reflecting strong seasonal variation. The minimum and maximum count of 1,264 and 43,864 respectively confirm that demand varies dramatically across the year. Edinburgh's climate is characterized by mild but cool conditions, with mean daily temperatures averaging 9.7°C and ranging from -8.1°C to 23°C, typical for a Scottish city. The wide spread in temperature variables, ranging from -14°C to 31°C, suggests that weather conditions vary considerably across the observation period and are likely to play an important role in explaining day-to-day variation in cycling demand.

Table 1: Summary statistics for daily cyclist count and temperature variables.

Variable	Mean	SD	Min	Median	Max
Daily Count	10815.5	4728.9	1264.0	10531.0	43864
Mean Temp (°C)	9.7	4.8	-8.1	9.8	23
Min Temp (°C)	5.9	5.0	-14.0	6.0	18
Max Temp (°C)	13.2	5.3	-3.0	13.0	31

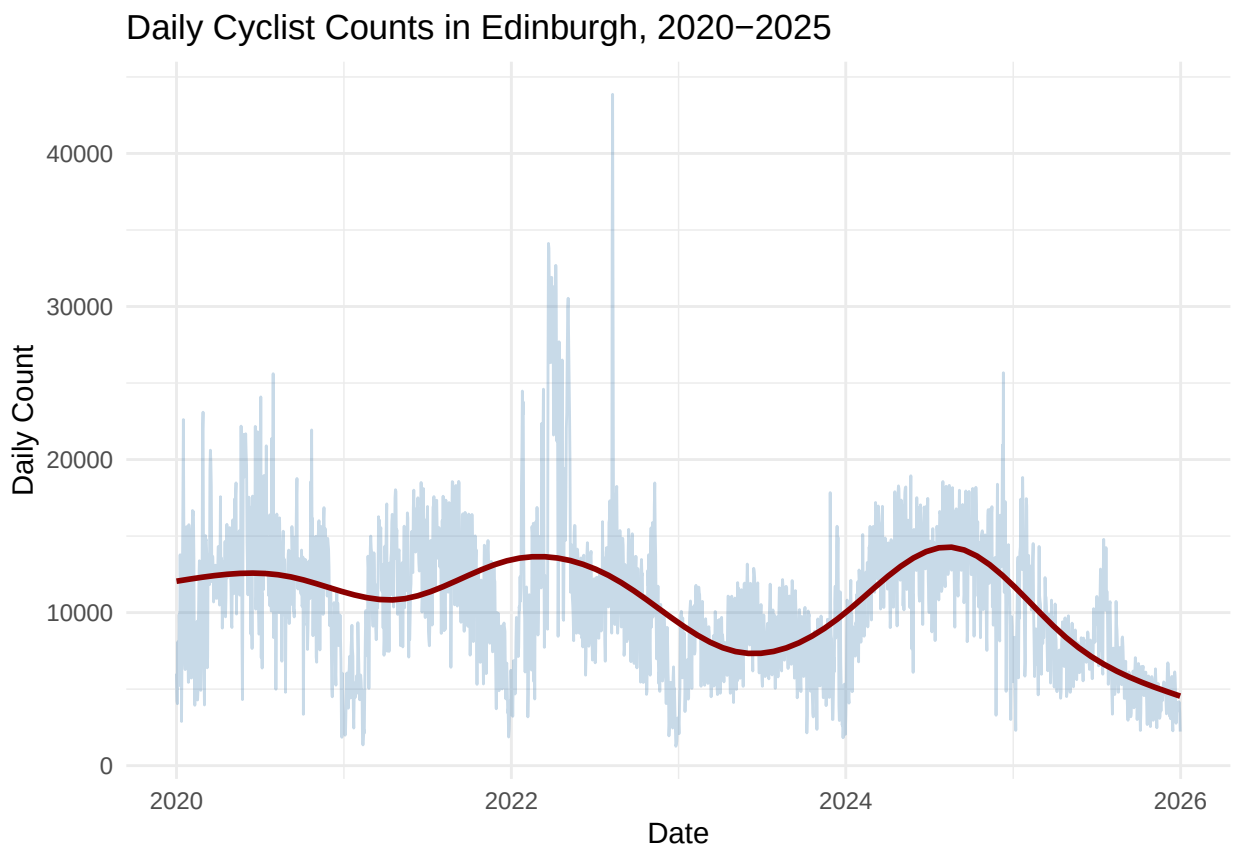


Figure 1: Time series of daily cyclist counts in Edinburgh (2020–2025) with a smoother.

Figure **??** shows the daily cyclist counts across the full observation period from 2020-2025, with a smoother overlaid to highlight the trend, filtering out day-to-day variability. We can see the obvious pattern that the daily count climbs every summer and fall back each winter, and this repeats every year. There is a visible dip around 2020-2021, which is unsurprising given the COVID-19 lockdowns during that period. After that, daily counts reach their highest levels around 2022-23. The drop at the end of the observation period coincides with the data ending in early 2026, making it difficult to distinguish a seasonal effect from a genuine trend change. Certain days stands out in count, especially one around 2022 that exceeds 40,000 cyclists a day, which probably reflects an unusually warm day.

Boxplots of count by month and by day of week

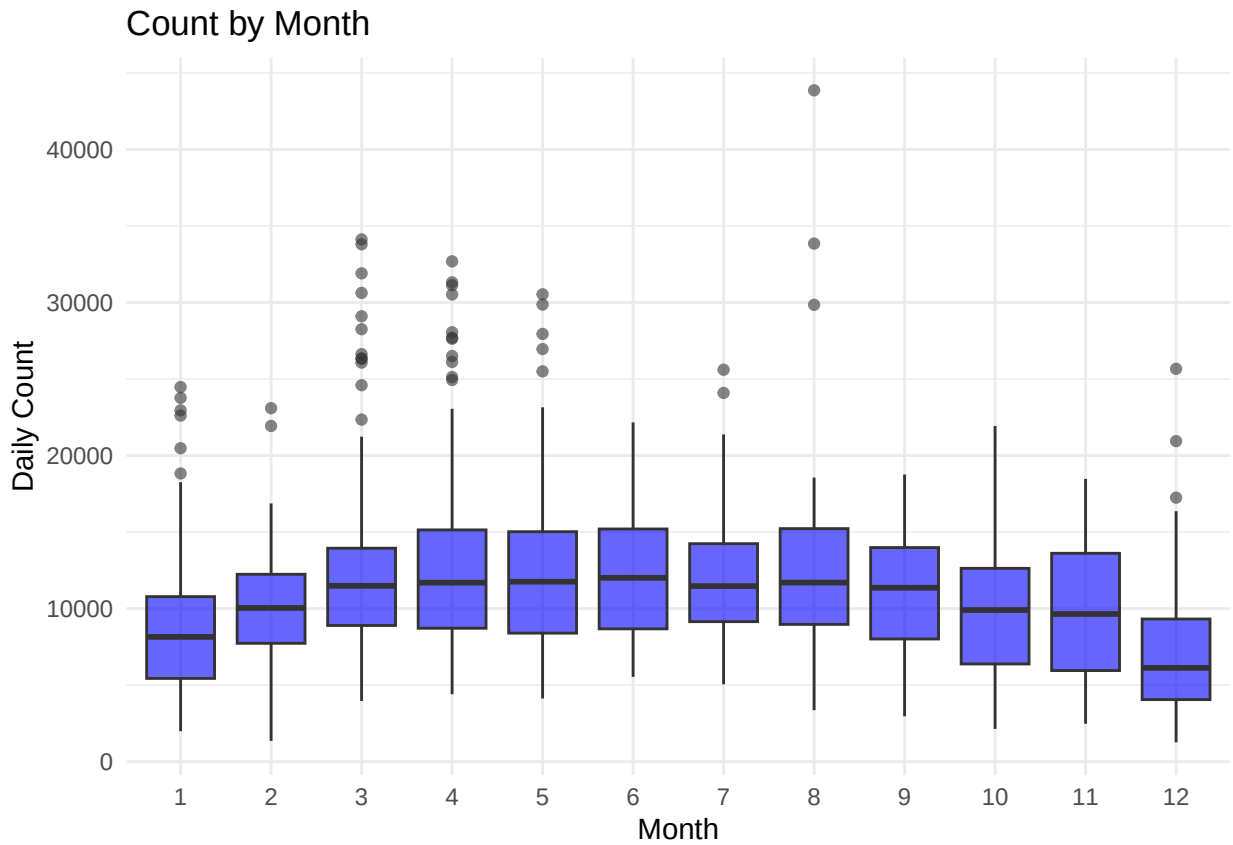


Figure 2: Distribution of daily cyclist counts by month.

?? confirms the seasonal pattern seen in the time series. Counts are lowest in January and December, with medians around 10,500-11,500 cyclists per day, rising steadily through spring and peaking between May and August. Winter months see roughly half the cycling activity of summer months, which has clear implications for seasonal infrastructure planning.

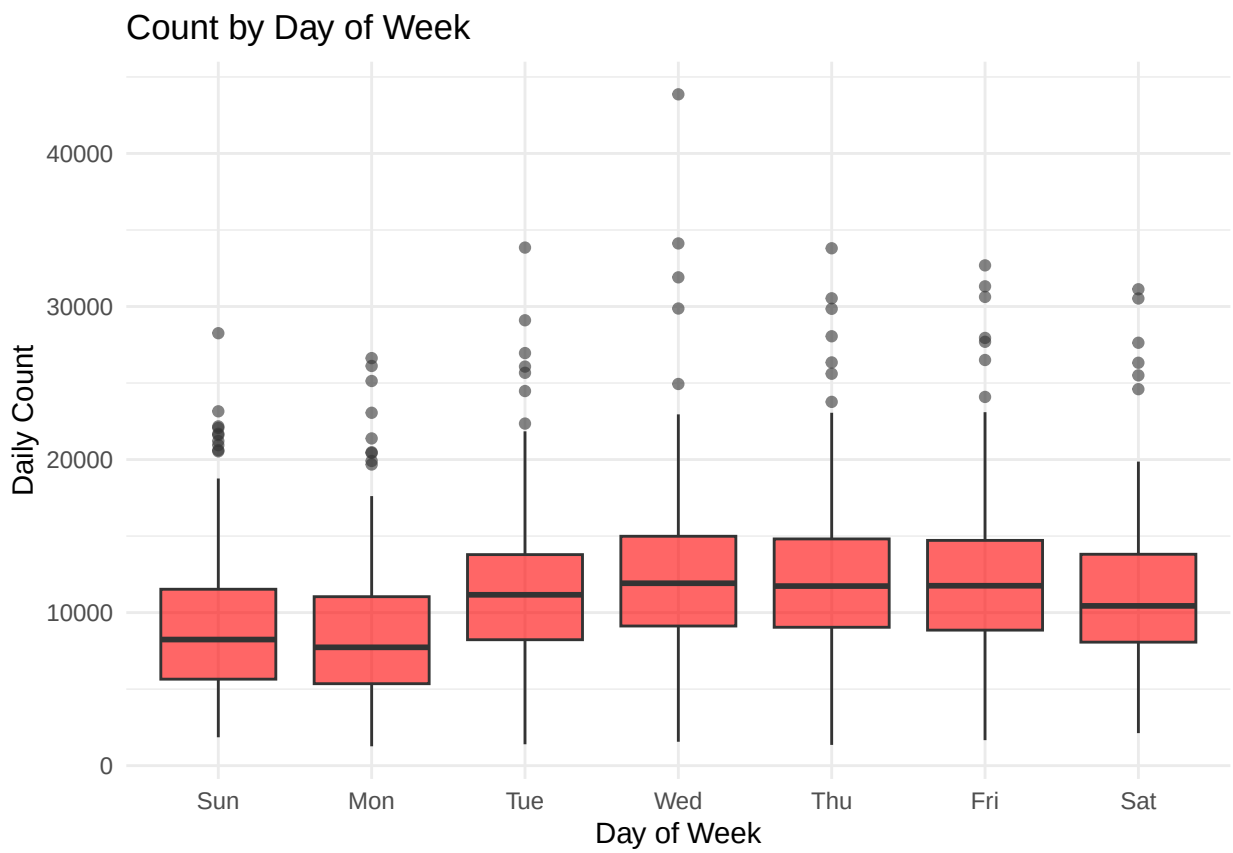


Figure 3: Distribution of daily cyclist counts by day of week

?? suggests that cycling demand is higher on weekdays than weekends, with Tuesday to Friday showing the highest medians of around 11,000-12,000 cyclists per day. Sunday and Monday stand out as the lowest, with medians closer to 8,000-9,000, suggesting that commuting is a major driver of cycling demand in Edinburgh.

Scatter plot of count vs temp_mean

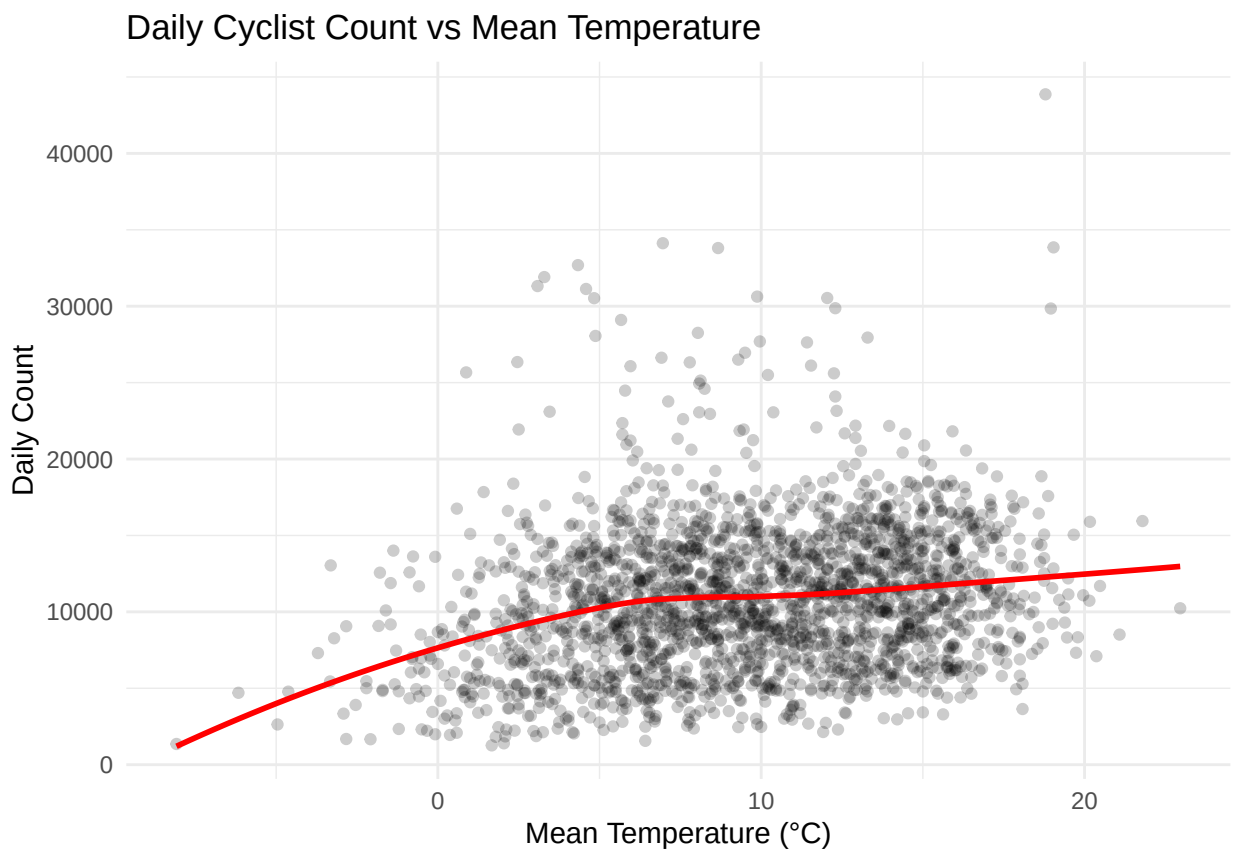


Figure 4: Daily cyclist count versus mean temperature

?? shows a clear positive but non-linear relationship between mean temperature and daily cyclist counts. Counts rise steeply at lower temperatures before leveling off above around 10–12°C, suggesting a quadratic relationship for temperature term. The spread of counts also increases with temperature, suggesting heteroscedasticity.

Showing relation between count and mean temperature during COVID and post COVID period

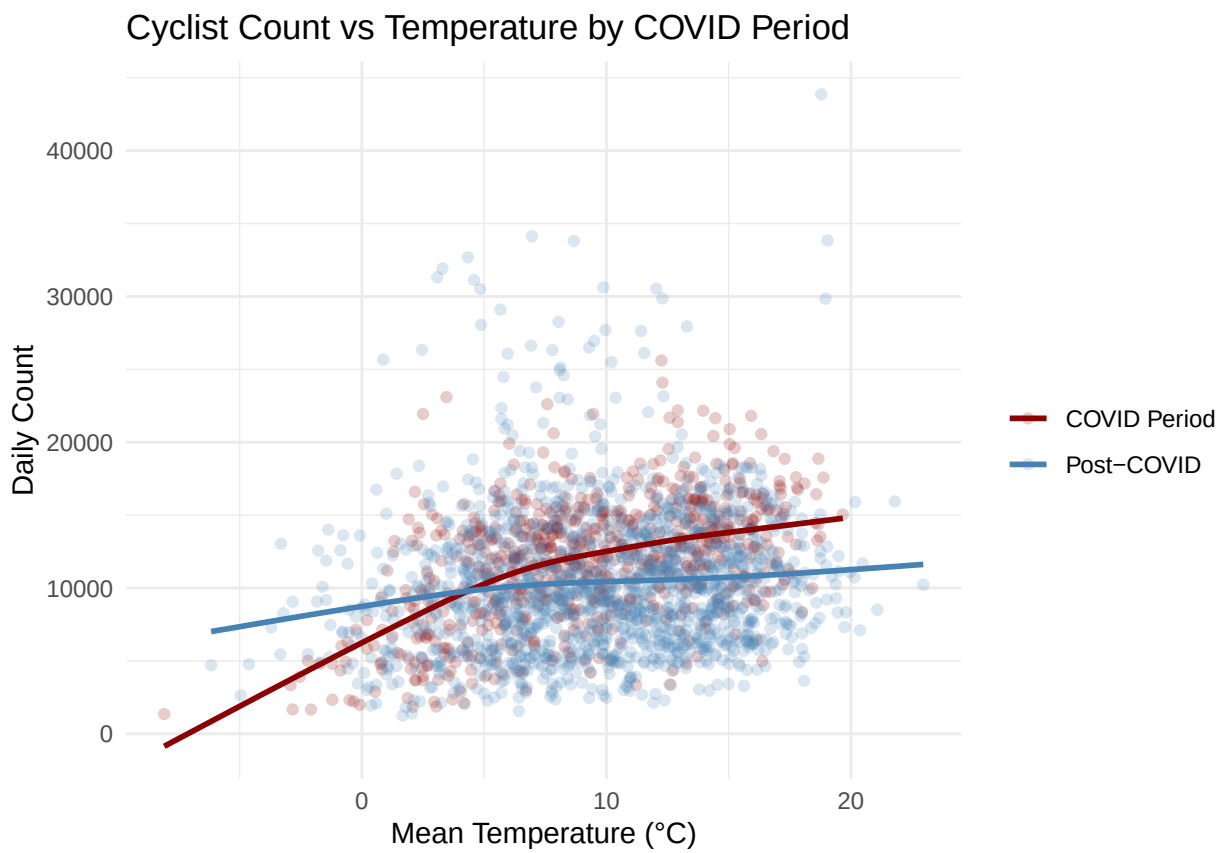


Figure 5: Daily cyclist count versus mean temperature by COVID and post-COVID period (2020–2025)

??reveals an interesting and somewhat counter intuitive pattern — cycling counts during the COVID period (2020-2021) were actually higher at warmer temperatures compared to post-COVID years, with the red smoother sitting well above the blue at temperatures above 10°C. This likely reflects that during lockdowns, cycling shifted from commuting to recreational use, with people more willing to cycle on pleasant days but much less likely to cycle in cold or wet conditions, explaining the steeper temperature sensitivity. At cold temperatures the pattern reverses, with post-COVID counts higher, consistent with commuters cycling regardless of weather. This difference in behaviour between the two periods suggests that a COVID indicator variable may improve model fit, and motivates its inclusion as a candidate extension in M3.

3 Baseline model M0

In all models, the response variable is $y_i = \text{count}_i$, the total daily cyclist count on day i. The common predictors are defined as follows: $x_{1i} = \text{temp_mean}_i$ (daily mean temperature in °C), $x_{2i} = \text{weekend}_i$ (binary weekend indicator), $x_{3i} = \text{trend}_i$ (days elapsed since 1 January 2020), $x_{4i} = \text{month}_i$ (month factor, reference: January), and $x_{5i} = \text{dow}_i$ (day of week factor, reference: Sunday). All models assume $\varepsilon_i \sim N(0, \sigma^2)$.

M0

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \sum_{m=2}^{12} \gamma_m \mathbf{1}(x_{3i} = m) + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2)$$

Table 2: Coefficients table for baseline model M0.

term	estimate	std.error	statistic	p.value
(Intercept)	10385.819	267.015	38.90	0
temp_mean	247.179	20.769	11.90	0
weekend	-1246.888	214.172	-5.82	0
month_num	-247.184	28.810	-8.58	0

The coefficients table for M0 shows the estimated coefficients for M0. All four predictors are highly significant ($p < 0.001$). Each 1°C increase in mean temperature is associated with approximately 247 additional cyclists per day with the other variables held fixed, while weekend days see around 1,247 fewer cyclists than weekdays, consistent with the commuter-driven pattern observed in the EDA. The negative month coefficient (-247) implies that daily count declines on average as the month increases, though this is a crude approximation since the relationship between month and demand is not linear.

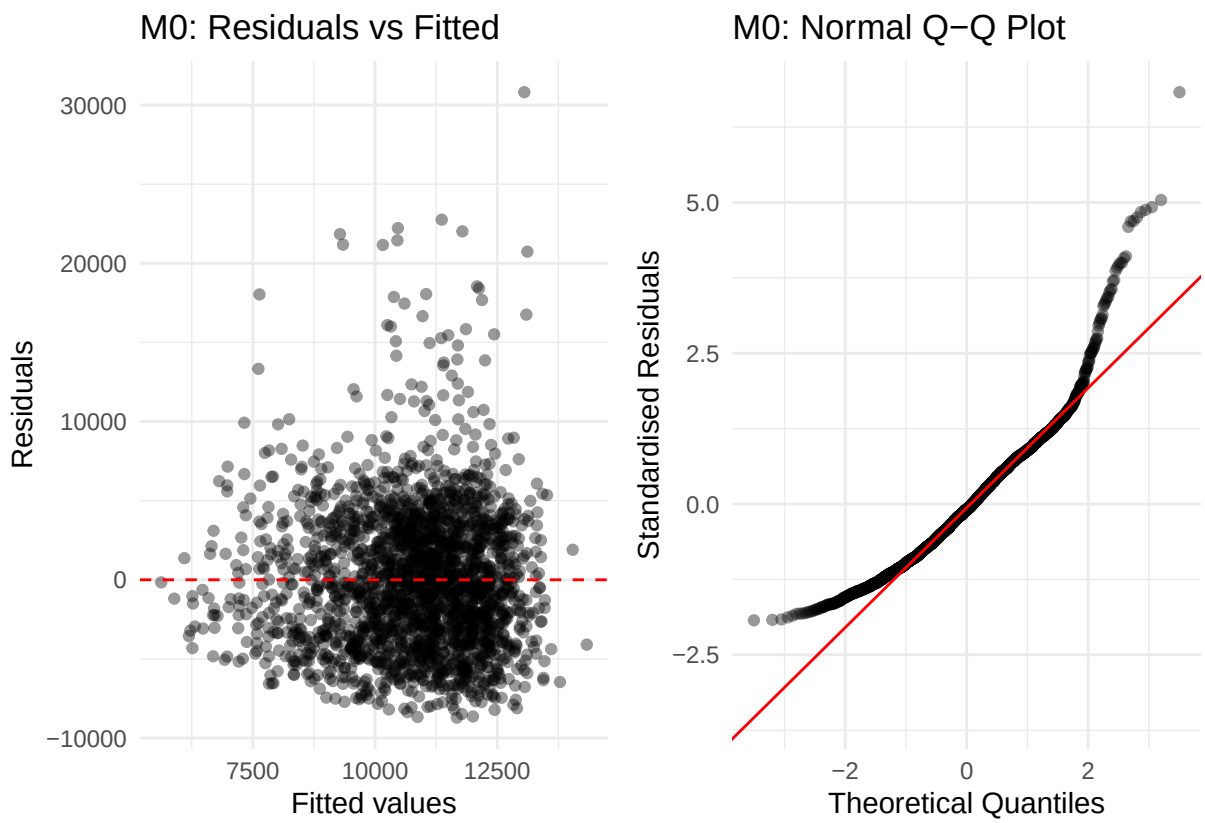


Figure 6: Diagnostics for M0

The residuals versus fitted plot shows that the spread of residuals grows noticeably with fitted values, indicating heteroscedasticity — a violation of the constant variance assumption. The QQ plot reveals a heavy right tail, with standardized residuals curving sharply above the reference line at high quantiles, reflecting the right-skewed distribution typical of non-negative count data. Both issues suggest that M0's Gaussian assumption is inadequate.

M0 captures the broad direction of key predictors — temperature, weekend effect, and seasonality but treats month as numeric, forcing a linear relationship across months that does not reflect the true seasonal pattern

seen in the EDA. It also omits day of week entirely and includes no long-term trend. These limitations motivate M1, which switches month to a factor, adds day of week as a factor, and introduces a linear trend term to capture drift in cycling demand over 2020–2025.

4 Proposed models M1–M3

4.1 Model M1: Trend and factor

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{3i} + \sum_{m=2}^{12} \gamma_m \mathbf{1}(x_{4i} = m) + \sum_{d=2}^{7} \delta_d \mathbf{1}(x_{5i} = d) + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2)$$

M1 extends the baseline model in three ways, each motivated by clear limitations identified in M0.

First, month is switched from a numeric predictor to a factor, giving each months its own intercept. The boxplot in Figure 2 shows that the relationship between month and demand is not monotonic — counts peak in summer and fall in winter but do not follow a straight line across month numbers 1 to 12, so forcing a single slope as M0 does is inappropriate.

Second, day of week is added as a factor. The boxplot in Figure 3 showed a clear difference between weekdays and weekends, but the weekend binary in M0 groups Saturday and Sunday together. Since Saturday behaves differently from Sunday — likely reflecting a mix of leisure and residual commuting trips — a full factor allows each day to have its own effect.

Third, a linear trend term is added, defined as the number of days since 1 January 2020. The time series in Figure 1 suggests that underlying demand has shifted over the 2020–2025 period, and without a trend term the model has no way to capture this drift, potentially biasing all other coefficient estimates

Diagnostics of M1

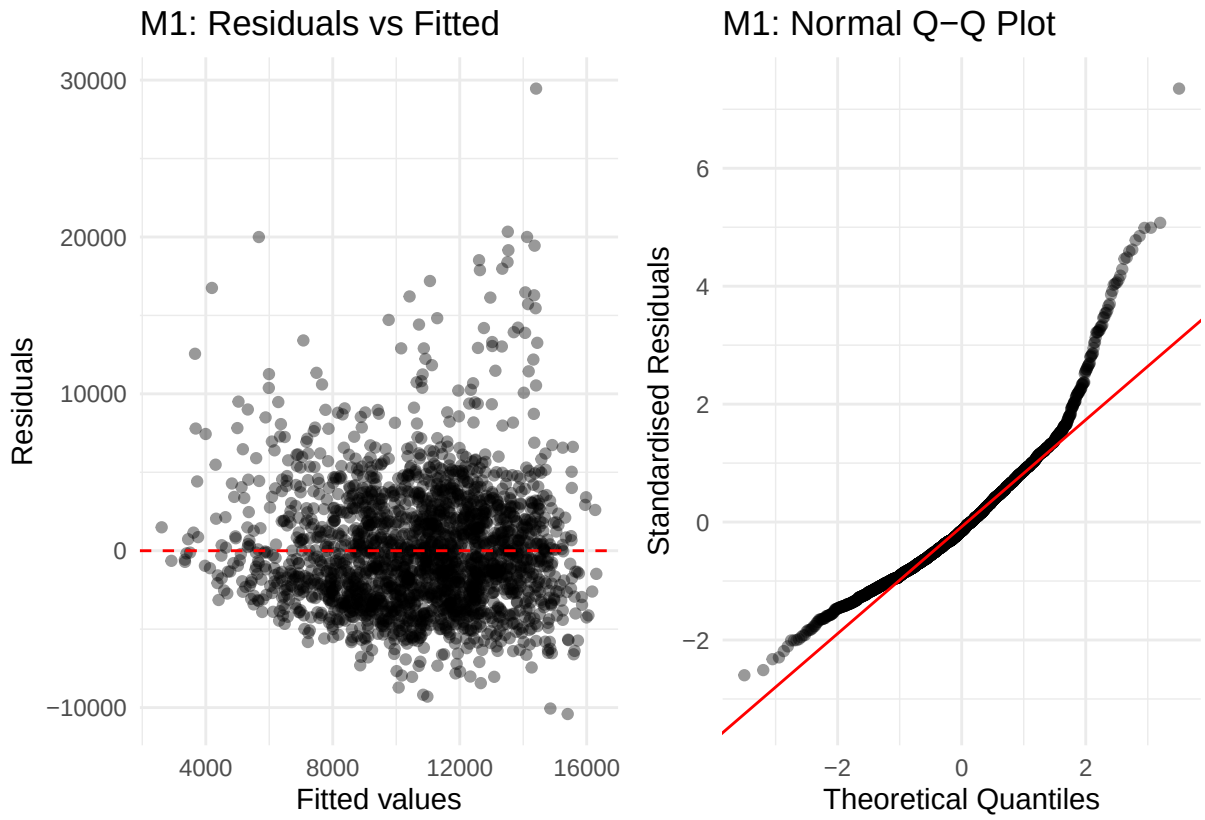


Figure 7: Diagnostics for M1

Compared to M0, M1 shows some improvement — the fitted values span a wider range, suggesting the additional predictors (trend, factor month, factor day of week) give the model more flexibility to capture variation in demand. However, the core issues persist. The residuals versus fitted plot still shows increasing spread at higher fitted values, indicating that heteroscedasticity remains unresolved. The QQ plot continues to show a pronounced right tail, with standardized residuals reaching above 6 at the upper end, reflecting the right-skewed nature of the count data that a Gaussian model struggles to accommodate. These issues motivate further extensions in M2 and M3.

4.2 Model M2: Nonlinear temperature

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{1i}^2 + \beta_3 x_{2i} + \beta_4 x_{3i} + \sum_{m=2}^{12} \gamma_m \mathbf{1}(x_{4i} = m) + \sum_{d=2}^{7} \delta_d \mathbf{1}(x_{5i} = d) + \varepsilon_i$$

M2 extends M1 by replacing the linear temperature term with a quadratic, adding both temp_mean and temp_mean^2 as predictors. The scatter plot in Figure 3 showed that the relationship between temperature and cycling demand flattens out above around 10°C rather than increasing at a constant rate, which a linear term cannot capture. Including the quadratic term allows the model to reflect this diminishing sensitivity to temperature at higher values, which is also physically plausible — once conditions are warm enough, further increases in temperature have a smaller effect on whether people choose to cycle.

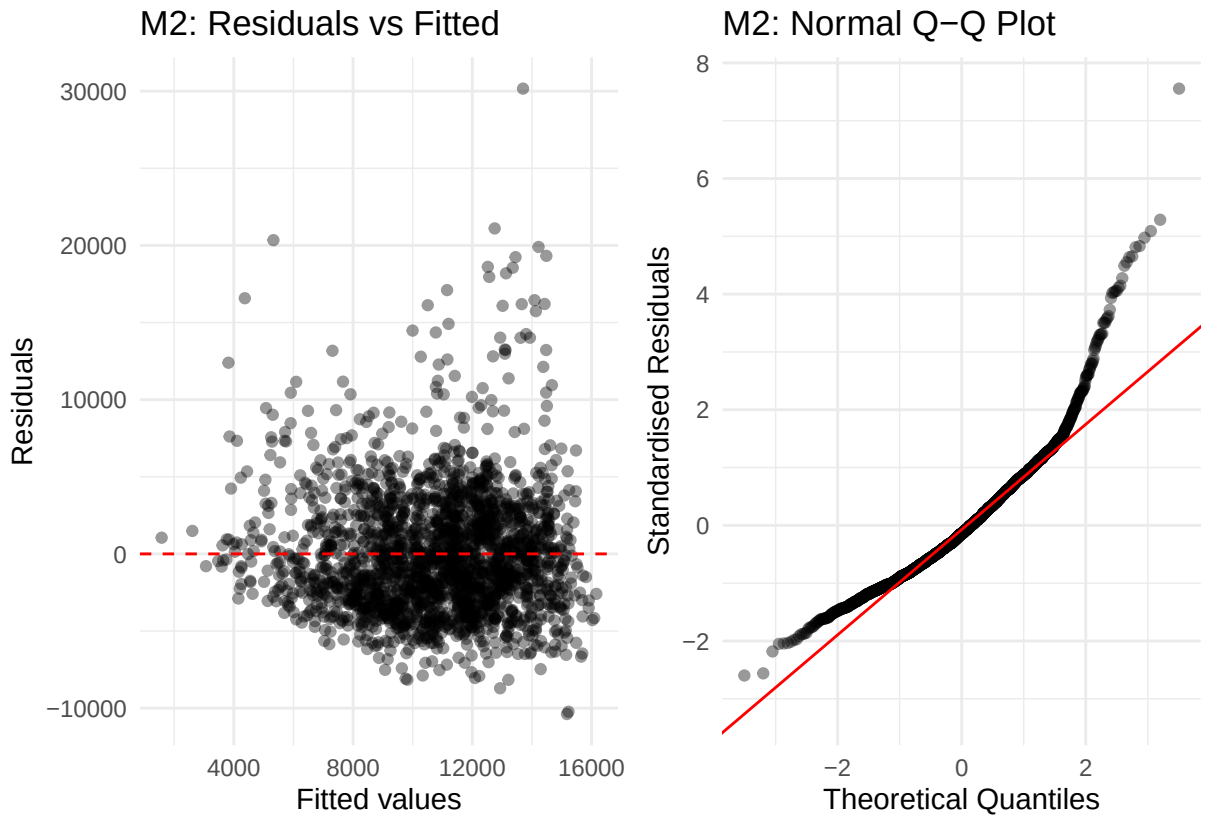


Figure 8: Diagnostics for M2

M2 shows marginal improvement over M1 — the residuals versus fitted plot appears slightly more evenly spread, suggesting the quadratic temperature term has captured some additional structure in the data. The lower tail of the QQ plot also follows the reference line more closely than in M1. However, the right tail remains heavily skewed, with standardized residuals exceeding 7 at the upper end, worse than M1. Heteroscedasticity also persists, with residual spread still growing at higher fitted values. The consistently heavy right tail across M0, M1, and M2 strongly suggests that a Gaussian model is fundamentally inappropriate for these count data, and motivates the log transformation of the response in M3.

4.3 Model M3: [Log-Transformed Response]

$$\log(y_i + 1) = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{1i}^2 + \beta_3 x_{2i} + \beta_4 x_{3i} + \sum_{m=2}^{12} \gamma_m \mathbf{1}(x_{4i} = m) + \sum_{d=2}^{7} \delta_d \mathbf{1}(x_{5i} = d) + \varepsilon_i$$

M3 keeps all predictors from M2 but log-transforms the response variable, fitting log(count + 1) rather than count directly. The diagnostic plots for **M0**, **M1**, and **M2** consistently showed two problems: a heavy right tail in the QQ plot and residual spread increasing with fitted values. Both are classic symptoms of right-skewed, non-negative count data being modelled with a Gaussian assumption. The log transformation directly addresses this — by compressing large values, it reduces skewness in the response and stabilizes variance, making the Gaussian assumption more plausible. The +1 is added before taking the log to handle any zero counts, since log(0) is undefined.

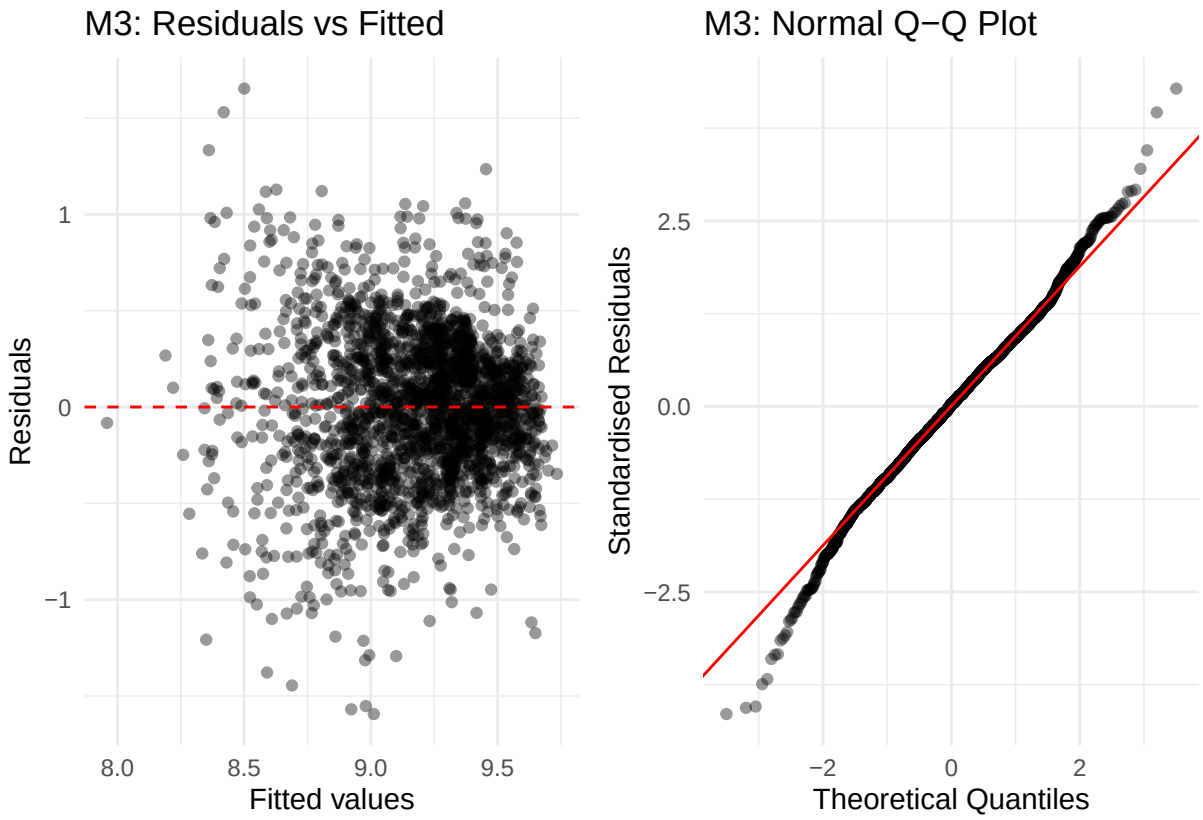


Figure 9: Diagnostics for M3

The diagnostic plots for **M3** show a marked improvement over all previous models. The residuals versus fitted plot no longer shows the growing spread seen in **M0** through M2 — residuals are now scattered fairly evenly around zero across the full range of fitted values, suggesting the log transformation has largely resolved the heteroscedasticity. The QQ plot is dramatically better, with standardised residuals following the reference line closely across almost the entire range, compared to the severe right tail deviation seen in M2 where residuals exceeded 7. The tails still deviate slightly at both ends, indicating some remaining non-normality, but this is a substantial improvement. Overall, the log transformation successfully addressed the two core issues identified in **M0** through **M2** — right skew and non-constant variance — making the Gaussian assumption considerably more plausible for M3 than for any of the earlier models.

5 Evaluation

5.1 Statistical Performance

Table 3: Table 1. Leave-one-year-out cross-validation scores for models M0–M3. Lower is better for all scores.

model	RMSE	MAE	DS	IS
M0	4740.9	3805.6	18.05	25630.5
M1	4602.3	3665.3	18.08	25391.0
M2	4587.6	3650.2	18.07	25277.9

model	RMSE	MAE	DS	IS
M3	4530.8	3598.5	-0.40	2.4

Table 1 shows a clear evolution in predictive performance as the models become more flexible. M0 performs worst across all four metrics. This indicates that a baseline model with numeric month and no trend or day-of-week factor is too limited. M1 improves on M0 given it allows separate month and day-of-week effects and adding a long-term trend. M2 gives a small further improvement by allowing a nonlinear temperature effect. M3 performs best on all four cross-validation measures, with the lowest RMSE, MAE, DS, and IS. This is natural as lower values are better for all four scores, and as all four metrics agree M3 is selected as the final model.

Table 4: Table 2. Monthly cross-validated RMSE and Dawid–Sebastiani score for the final model M3.

Month	RMSE	DS
Jan	4377.6	-0.14
Feb	3973.2	-0.30
Mar	5886.6	-0.62
Apr	6485.7	-0.25
May	5035.7	-0.77
Jun	3909.0	-1.08
Jul	3485.3	-1.24
Aug	5301.0	-0.63
Sep	4270.3	-0.40
Oct	4086.3	-0.26
Nov	4310.5	-0.10
Dec	4381.3	0.92

Table 2 indicates that the final model performance varies noticeably throughout the year. RMSE is lowest in July (3485.3), with June (3909.0) and February (3973.2) also showing relatively strong performance, indicating that cyclist demand is predicted most accurately in these months. In contrast, RMSE is highest in April (6485.7) and March (5886.6), suggesting that demand is hardest to predict in early spring. The Dawid–Sebastiani score is also strongest in summer, reaching its lowest value in July (-1.24) and remaining low in June (-1.08), but it worsens sharply in December (0.92), indicating poorer calibration of the predictive distribution at the start of winter. In essence, from Table 2 we can conclude that although M3 performs the best overall, its predictive reliability is not uniform accross the calendar year, where it encounter the most difficulty in early spring and December.

5.2 Practical Performance

5.2.1 5.1 Temperature effects and nonlinearity

This subsection compares three nonlinear temperature specifications based on mean, minimum, and maximum daily temperature for our M2 model. Our ambition is to determine which temperature measure gives the best predictive performance. We then analyse the marginal effects of the fitted nonlinear at two representitive chosen temperature variables, 5°C and 15°C in this case.

Table 5: Comparison of nonlinear temperature specifications in the M2 model family.

temp_var	RMSE	DS	IS
temp_max	4599.1	18.09	25317.1
temp_mean	4587.6	18.07	25277.9
temp_min	4574.7	18.06	25010.3

Between the three nonlinear temperature specification we can see the model utilizing temp_min performed the best as it had the lowest RMSE score of 4574.7, DS score og 18.06 and IS score of 25010.3. However, the DS score was effectiveley identical between al three only different by 0.03 at most.This indicates that, within this model class, temp_min provides the strongest predictive performance.

This is a realistic conclusion to make. Majority of commuters will likely check their weather apps and decide whether they will bike based off the coldest conditions during the day. This is particularly true for morning commuters as this is typically the coldest period of time as well. Minimum daily temperature may therefore be more relevant than average or maximum temperature in determining whether people choose to cycle at all.

Table 6: Estimated marginal effect of a 1°C increase using the chosen temperature variable.

Temperature	Marginal.effect..cyclists.per..1C.
5C	53.9
15C	-250.2

For the chosen model, the estimated effect of temperature depends on the temperature level because the model includes a quadratic term. At 5°C, a 1°C increase is associated with approximately 53.9 additional cyclists per day. At 15°C, the fitted model implies that a 1°C increase is associated with approximately 250.2 fewer cyclists per day.

The positive marginal effect at 5°C is intuitive, as slightly milder cold days are likely to encourage more cycling. The negative marginal effect at 15°C should be interpreted more cautiously. Instead of implying that warmer weather necessarily reduces cycling, as 15°C is a pleasant biking temperature, and would intuitively imply either an increase rather or a slight decrease over a significant decrease. This therefore more likely reflects the curvature imposed by the quadratic specification and the fact that the relationship appears to flatten at higher temperatures. Because the model does not include a temperature × season interaction, this temperature effect is assumed to be the same across the year, which is a limitation if weather sensitivity differs between seasons.

5.2.2 5.2 Seasonal prediction accuracy

6 Discussion and limitations

The purpose of this subsection is to assess how reliably the final model predicts cycling demand across the calendar year. Using the monthly cross-validation results from Table 2, we identify which months are predicted best and worst and consider what this implies for transport planning in Edinburgh.

Table 7: Monthly cross-validated RMSE and Dawid–Sebastiani score for the final model M3.

Month	RMSE	DS
Jan	4377.6	-0.14
Feb	3973.2	-0.30
Mar	5886.6	-0.62
Apr	6485.7	-0.25
May	5035.7	-0.77
Jun	3909.0	-1.08
Jul	3485.3	-1.24
Aug	5301.0	-0.63
Sep	4270.3	-0.40
Oct	4086.3	-0.26
Nov	4310.5	-0.10
Dec	4381.3	0.92

As discussed in the Statistical Performance section, Table 2 shows that the predictive performance of the final model varies across the year. Prediction accuracy is strongest in summer, especially in June and July, where both RMSE and DS are lowest. The model performs worst in March and April on RMSE, while December has the highest DS, indicating poorer calibration of the predictive distribution. M3 is the best overall model, yet its performance is not equally reliable across all months.

A plausible transport-planning interpretation is that cycling demand is easier to predict in summer because behaviour is more stable and weather conditions are more consistently favourable. In winter, demand may also be somewhat easier to model for a different reason: for many people, conditions are already too cold or unpleasant for cycling, so small changes in temperature are less likely to affect the decision to bike at all. This reduces behavioural variation and makes demand more predictable. By contrast, early spring is likely to be harder to predict because it is a transition period. As temperatures begin to improve, people may differ much more in the point at which they feel conditions are "good enough" to start cycling again. This creates a mental and behavioural threshold that varies across individuals, making aggregate demand more sensitive to small changes in weather and therefore harder to model accurately. The relatively poor performance in March, April, and December is therefore likely to reflect both model limitations and data limitations: the model omits factors such as rainfall, wind, holidays, and special events, while the available predictors may not fully explain abrupt seasonal changes in cycling behavior.

6.0.1 5.3 Long-term trend and extrapolation

This subsection examines the long-term trend in cycling demand implied by the fitted models and considers what that trend means for transport planning. First the estimated annual change is reported, and then its statistical significance, followed by an assessment of whether the trend is robust across M1, M2, and M3, and finally checks whether the COVID period materially affects the estimate, and finaly uses M3 to extrapolate demand forward to 2035 under fixed summer weekday conditions.

Table 8: Table 3. Trend coefficients from models M1–M3.

Model	Daily.change..cyclists.day.	Annual.change..cyclists.year.
M1	-1.895	-692
M2	-1.872	-683
M3	-2.152	-786

Table 9: Table 4. Statistical significance of the trend coefficient in models M1–M3

Model	Estimate	Std_Error	t_value	p_value
M1	-1.8945	0.1392	-13.61	0
M2	-1.8724	0.1390	-13.47	0
M3	-0.0002	0.0000	-13.92	0

Table 10: Table 5. Annual growth rate expressed relative to the predicted count on 1 January 2020.

Model	Predicted_start_count	Daily_change	Annual_change	Growth_percent
M1	12222.3	-1.895	-692	-5.66
M2	12462.5	-1.872	-683	-5.48
M3	11512.3	-2.152	-786	-6.82

Table 3 and the significance table show that all three models estimate a negative trend in daily cycling demand over the observed period. The trend in M1 corresponds to approximately 692 fewer cyclists per year, in M2 to 683 fewer cyclists per year, and in M3 to about 786 fewer cyclists per year. Expressed relative to the predicted count on 1 January 2020, this corresponds to annual percentage changes of -5.66% in M1, -5.48% in M2, and -6.82% in M3. The trend is statistically significant in all three models, with large negative t-values and p-values effectively equal to zero.

The estimated trend is relatively consistent across M1, M2, and M3. All three models imply an approximate decline of 680–790 cyclists per year. This suggests that the broad direction and magnitude of the trend are reasonably robust to modelling choices. This consistency is useful for planning because it indicates that the conclusion of declining demand is not being driven by one particular specification alone. At the same time, the slightly more negative estimate in M3 shows that allowing for skewness and non-constant variance can shift the implied rate of change somewhat, even if the overall conclusion remains the same.

Table 11: Table 6. Comparison of M3 trend estimates with and without a COVID indicator.

Model	Daily_change	Annual_change
M3	-2.152	-786
M3 + COVID	-2.904	-1060

The COVID period seems to be of significance for the interpretation of the long-term trend. With the COVID indicator for model M3, the estimated daily change becomes more negative, shifting from approximately -2.152 cyclists/day to -2.904 cyclists/day. This corresponds to an annual change from about -786 to -1060 cyclists/year, suggesting that the simple linear trend in M3 is partly absorbing the unusual behaviour of the pandemic years. In other words, a single linear trend fitted across 2020–2025 may be too basic, blending together the COVID dip, subsequent recovery, and later pattern into one average slope. This means the trend should be interpreted cautiously rather than treated as a stable structural decline.

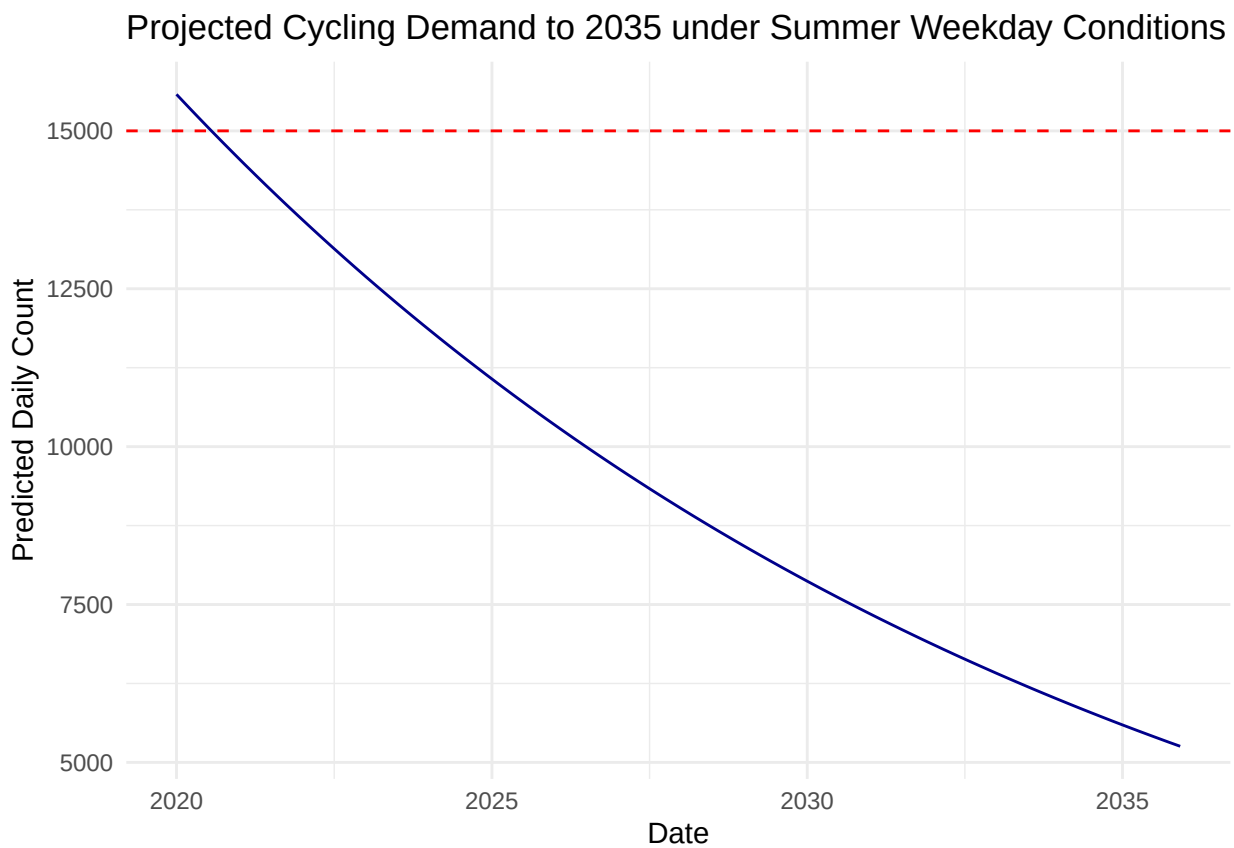


Table 12: Table 7. Earliest projected date at which predicted summer weekday demand exceeds 15,000 cyclists/day.

date	pred_count
2020-01-01	15578.52

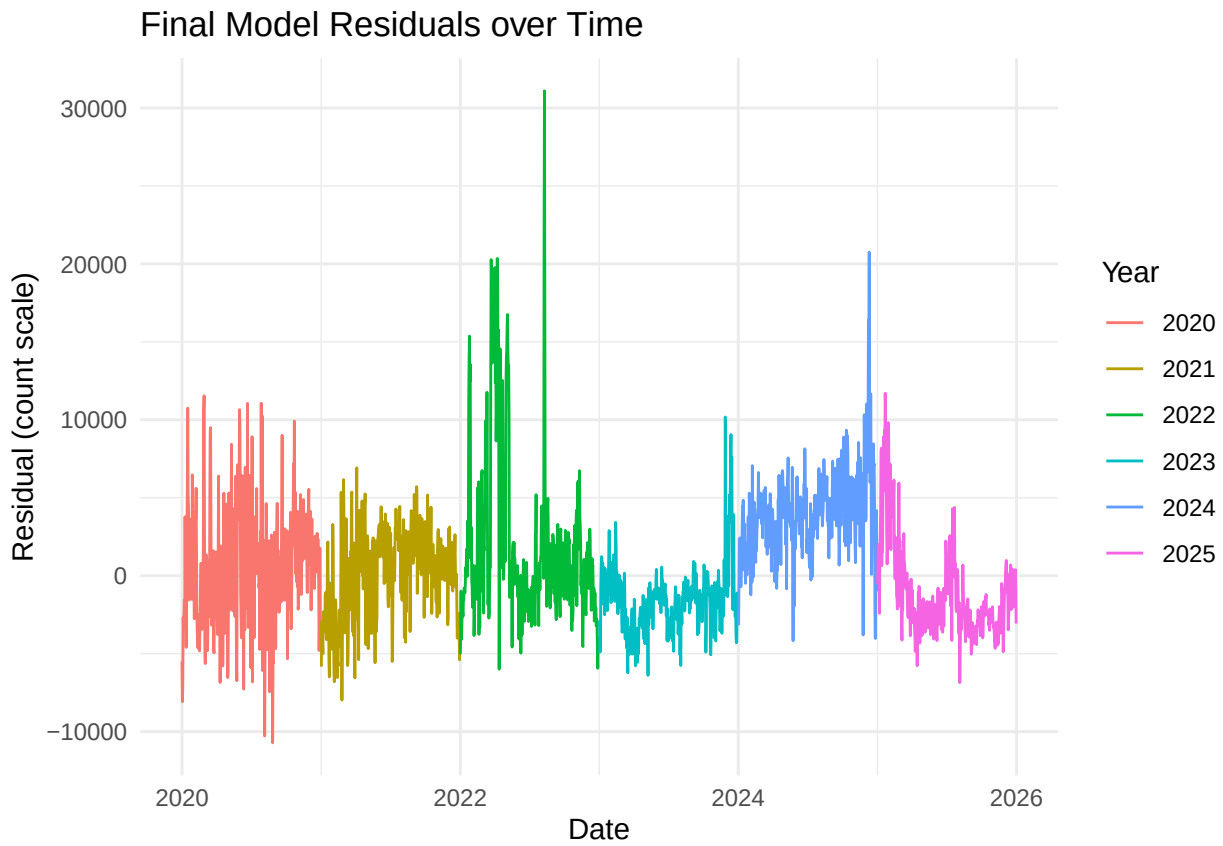
The extrapolation based on M3 projects declining, rather than increasing, summer weekday cycling demand through to 2035. Using the fitted model we can see that the predicted demand is already above the 15,000 cyclists/day threshold at the start of the period, starting at an estimated value of 15,579 on 1 January 2020. It then falls steadily, where in 15 years, the estimated value for cyclists/day is 5,300. This means that, at the current fitted rate, the model does not predict future growth toward the threshold; instead, it suggests that demand moves away from it over time.

This kind of long-run extrapolation should be treated with caution for at least two reasons. First, the fitted linear trend is estimated over a short period, specfically the COVID-19 disruption, where many were forced to stay indoors, hence it may not represent the true long-term path of cycling demand. Secondly, the projection holds weather and seasonal conditions fixed and ignores other factors that could substantially alter future demand. Notably as infrastructure investment, policy changes, fuel prices, or changes in population and commuting patterns. For these reasons, the extrapolation is useful as a simple model-based scenario, but not as a reliable long-term forecast.

6.0.2 5.4 Identifying a poorly-fit period

This subsection examines the residuals of the final model over time in order to identify a specific period in which the model performs notably poorly. The aim is to determine whether the model systematically

over- or under-predicts during that period, and to consider what that implies from both a statistical and transport-planning perspective.



The residual-over-time plot identifies periods where the final model appears to fit particularly poorly. This can be caused either by residuals that are unusually large in magnitude or because they are persistently positive or negative over a sustained interval.

Table 13: Table 8. Months with the largest RMSE under the final model.

year_month	mean_residual	RMSE
2022-04	8934.418	11317.815
2022-03	6851.281	10416.665
2024-12	6009.515	7866.465
2022-08	3602.210	7861.138
2022-05	4191.181	7662.392
2024-10	5446.711	5885.668
2025-01	4548.762	5858.492
2022-01	1747.584	5774.558
2024-11	4927.126	5598.794
2024-09	4922.280	5233.538

Table 8 shows the months in which the final model performs worst, ranked by period-specific RMSE. This table will therefore help indicate the exact month with worst performance.

April 2022 stands out as the single poorest-fitted month. It has the largest month-specific RMSE in the

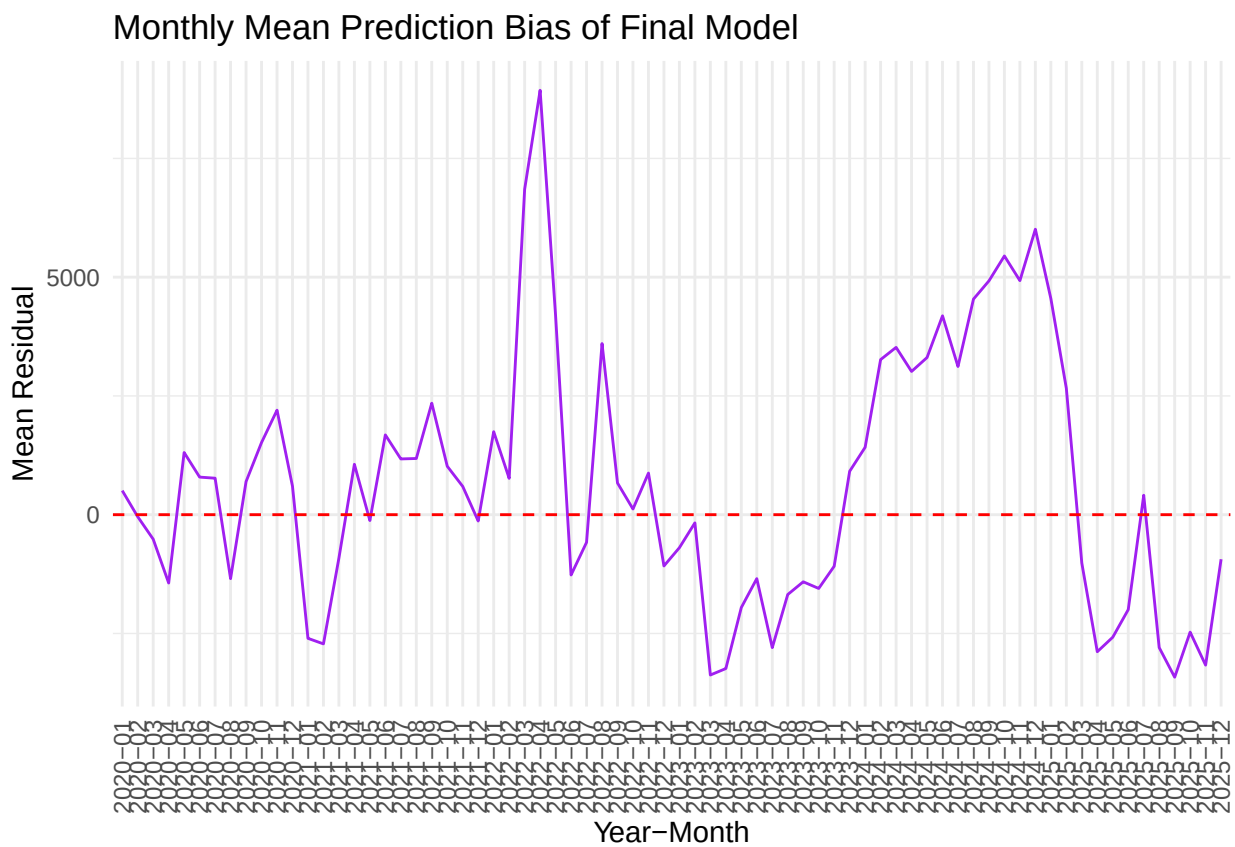
table, equal to 11317.8, and a positive mean residual of 8934.4. This indicates that the model under-predicts cyclist demand consistently during this period of time rather than making a few isolated large errors.

This implies April 2022 was a period of unusually high cycling demand that is not fully explained by the variables included in the model. The residual plot shows a large grouping of positive residuals during spring and summer 2022. This suggests that the model may be missing some factor that lifted counts above their usual seasonal level.

From a transport-planning perspective, this reflect changes in travel behavior, especially if cycling demand temporarily increased because of favorable or unfavorable conditions. There was potentially a post-pandemic behavioural shifts, where increased free movement was taken advantage. On the other hand, there may have been external factors such as fuel prices. The Ukraine-Russia war started in Feb.2022 and had resulted in significant increases in the prices of crude oil. Statistically, the poor fit also reflects model limitations: the final model includes temperature, seasonality, day of week, and trend, but omits potentially important short-term drivers such as rainfall, wind, holidays, oil prices and special events. April 2022 therefore appears to be a period where both the data limitations and the model specification contribute to poor predictive performance.

6.0.3 5.5 Original Diagnostic Plot

This subsection presents an original plot that is not a standard diagnostic, designed to reveal an interesting feature of the final model that is not visible from standard diagnostics plots. This plot tracks the monthly mean residual over time, showing whether the model systematically over- or under-predicts demand in different periods



The plot shows the monthly mean residual of the final model over time. Positive values indicate systematic under-prediction and negative values indicate systematic over-prediction. Multiple periods are visibly out,

particularly parts of 2022 and late 2024, where the model persistently under-predicts cycling demand, while much of 2025 shows negative mean residuals, indicating over-prediction. A transport planner should conclude that the model captures overall demand patterns reasonably well, but it struggles to track shifts in underlying behaviour across time. In particular, some periods appear to involve sustained changes in demand that are not fully explained by temperature, seasonality, day of week, and trend alone.

This plot is useful because it highlights regional bias rather than just overall error magnitude. Unlike RMSE or QQ plots, it shows when the model is persistently wrong in one direction, which is especially important for planning. A model that repeatedly under-predicts demand during certain periods may lead planners to underestimate pressure on cycling infrastructure, while persistent over-prediction could lead to over-allocation of resources.

7 Discussion and limitations

8 Appendix: Code

All analysis code is in `code.R` and is reproduced below.

```
# ============================================================
# Project 2: Modelling daily cycling demand in Edinburgh
# Authors: [Karthik Sutheeshkumar], [Kiet Lam], [Leon Godtfredsen]
# ============================================================

#
# Place the code needed in the Report_project02.Rmd, including documentation.
#

# 0. Packages
library(ggplot2); library(dplyr); library(tidyr)
library(lubridate); library(knitr); library(kableExtra)
library(patchwork); library(broom)

# 1. Load data
load('cycle_daily_df.Rdata')

# 2. Data Wrangling
cycle_daily_df <- cycle_daily_df %>%
  mutate(
    month_num = month,
    month = factor(month, levels = 1:12, ordered = TRUE),
    dow = factor(dow, levels = c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"),
                 ordered = TRUE),
    trend = as.integer(date - as.Date("2020-01-01"))
  )

cycle_cv_df <- cycle_daily_df %>%
  mutate(month = as.numeric(month))

# 3. Model Fitting
# Note: Use factor(month) in formulas for M1-M3
model_formulas <- list(
  M0 = count ~ temp_mean + weekend + month_num,
```

```r
  M1 = count ~ temp_mean + weekend + trend + factor(month) + factor(dow),
  M2 = count ~ weekend + trend + factor(month) + factor(dow) +
    temp_mean + I(temp_mean^2),
  M3 = log(count + 1) ~ weekend + trend + factor(month) + factor(dow) +
    temp_mean + I(temp_mean^2)
)

m0 <- lm(model_formulas[["M0"]], data = cycle_daily_df)
m1 <- lm(model_formulas[["M1"]], data = cycle_daily_df)
m2 <- lm(model_formulas[["M2"]], data = cycle_daily_df)
m3 <- lm(model_formulas[["M3"]], data = cycle_daily_df)

# 4. Cross-Validation Functions
calc_scores <- function(y, mu, sigma, alpha = 0.05) {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 + predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS here

  RMSE <- sqrt(mean((y-mu)^2))
  MAE <- mean(abs(y-mu))
  DS <- (y-mu)^2/sigma^2 + 2 * log(sigma)
  l  <- mu - qnorm(1 - alpha/2) * sigma
  u  <- mu + qnorm(1 - alpha/2) * sigma
  IS <- (u-l) + (2/alpha)*pmax(l-y,0) + (2/alpha)*pmax(y-u,0)
  return(list(RMSE = RMSE, MAE = MAE, DS = DS, IS = IS))

}

# 5. Leave-One-Year-Out CV Loop
years <-unique(cycle_cv_df$year)

cv_results <- bind_rows(lapply(names(model_formulas), function(model_name) {
  bind_rows(lapply(years, function(yr) {

    # Splitting into train and test
    test_df  <- cycle_cv_df %>% filter(year == yr)
    train_df <- cycle_cv_df %>% filter(year != yr)

    # Fitting model on training data
    fit <- lm(model_formulas[[model_name]], data = train_df)

    # Predicting on test data
    pred  <- predict(fit, newdata = test_df, se.fit = TRUE)
    mu    <- pred$fit
    sigma <- sqrt(summary(fit)$sigma^2 + pred$se.fit^2)

    # Observed values from test_df
    y <- test_df$count
```

```r
    if (model_name == "M3") {
      y_log <- log(test_df$count + 1)  # log scale for DS and IS
      y     <- test_df$count            # count scale for RMSE and MAE

      mu_log <- mu                      # log scale
      mu_count <- exp(mu) - 1           # count scale

      sigma_log <- sigma                # log scale

      # RMSE and MAE on count scale
      RMSE <- sqrt(mean((y - mu_count)^2))
      MAE  <- mean(abs(y - mu_count))

      # DS and IS on log scale
      scores_log <- calc_scores(y_log, mu_log, sigma_log)

      return(data.frame(
        model = model_name,
        year  = yr,
        RMSE  = RMSE,
        MAE   = MAE,
        DS    = mean(scores_log$DS),
        IS    = mean(scores_log$IS)
      ))
    }

    # CV score predictions against observed values
    CV_scores <- calc_scores(y, mu, sigma)

    data.frame(
      model = model_name,
      year  = yr,
      RMSE  = CV_scores$RMSE,
      MAE   = CV_scores$MAE,
      DS    = mean(CV_scores$DS),
      IS    = mean(CV_scores$IS)
    )
  }))
}))

# Table 1: average scores across all held-out years
cv_table1 <- cv_results %>%
  group_by(model) %>%
  summarise(
    RMSE = round(mean(RMSE), 1),
    MAE  = round(mean(MAE), 1),
    DS   = round(mean(DS), 2),
    IS   = round(mean(IS),1),
    .groups = "drop"
  )

print(cv_table1)
```

```r
# 6. CV by Month

# Empty data frame to store observation-level CV predictions
cv_month_data <- cycle_cv_df %>%
  mutate(
    mean_M0 = NA_real_, sd_M0 = NA_real_,
    mean_M1 = NA_real_, sd_M1 = NA_real_,
    mean_M2 = NA_real_, sd_M2 = NA_real_,
    mean_M3 = NA_real_, sd_M3 = NA_real_
  )

# Fill in leave-one-year-out predictions for each model
for (yr in years) {

  train_data <- cycle_cv_df %>% filter(year != yr)
  test_ind   <- which(cycle_cv_df$year == yr)
  test_data  <- cycle_cv_df[test_ind, , drop = FALSE]

  # M0
  fit_M0 <- lm(model_formulas[["M0"]], data = train_data)
  pred_M0 <- predict(fit_M0, newdata = test_data, se.fit = TRUE)
  cv_month_data[test_ind, "mean_M0"] <- pred_M0$fit
  cv_month_data[test_ind, "sd_M0"] <- sqrt(pred_M0$se.fit^2 + summary(fit_M0)$sigma^2)

  # M1
  fit_M1 <- lm(model_formulas[["M1"]], data = train_data)
  pred_M1 <- predict(fit_M1, newdata = test_data, se.fit = TRUE)
  cv_month_data[test_ind, "mean_M1"] <- pred_M1$fit
  cv_month_data[test_ind, "sd_M1"] <- sqrt(pred_M1$se.fit^2 + summary(fit_M1)$sigma^2)

  # M2
  fit_M2 <- lm(model_formulas[["M2"]], data = train_data)
  pred_M2 <- predict(fit_M2, newdata = test_data, se.fit = TRUE)
  cv_month_data[test_ind, "mean_M2"] <- pred_M2$fit
  cv_month_data[test_ind, "sd_M2"] <- sqrt(pred_M2$se.fit^2 + summary(fit_M2)$sigma^2)

  # M3
  fit_M3 <- lm(model_formulas[["M3"]], data = train_data)
  pred_M3 <- predict(fit_M3, newdata = test_data, se.fit = TRUE)
  cv_month_data[test_ind, "mean_M3"] <- pred_M3$fit
  cv_month_data[test_ind, "sd_M3"] <- sqrt(pred_M3$se.fit^2 + summary(fit_M3)$sigma^2)
}

# Compute scores for each observation
score_month <- cv_month_data %>%
  mutate(
    se_M0 = (count - mean_M0)^2,
    ds_M0 = (count - mean_M0)^2 / sd_M0^2 + 2 * log(sd_M0),
    lwr_M0 = mean_M0 - qnorm(0.975) * sd_M0,
    upr_M0 = mean_M0 + qnorm(0.975) * sd_M0,
    is_M0 = (upr_M0 - lwr_M0) +
      (2 / 0.05) * pmax(lwr_M0 - count, 0) +
      (2 / 0.05) * pmax(count - upr_M0, 0),
```

```r
    se_M1 = (count - mean_M1)^2,
    ds_M1 = (count - mean_M1)^2 / sd_M1^2 + 2 * log(sd_M1),
    lwr_M1 = mean_M1 - qnorm(0.975) * sd_M1,
    upr_M1 = mean_M1 + qnorm(0.975) * sd_M1,
    is_M1 = (upr_M1 - lwr_M1) +
      (2 / 0.05) * pmax(lwr_M1 - count, 0) +
      (2 / 0.05) * pmax(count - upr_M1, 0),

    se_M2 = (count - mean_M2)^2,
    ds_M2 = (count - mean_M2)^2 / sd_M2^2 + 2 * log(sd_M2),
    lwr_M2 = mean_M2 - qnorm(0.975) * sd_M2,
    upr_M2 = mean_M2 + qnorm(0.975) * sd_M2,
    is_M2 = (upr_M2 - lwr_M2) +
      (2 / 0.05) * pmax(lwr_M2 - count, 0) +
      (2 / 0.05) * pmax(count - upr_M2, 0),

    mean_M3_count = exp(mean_M3) - 1,
    se_M3 = (count - mean_M3_count)^2,
    ds_M3 = (log(count + 1) - mean_M3)^2 / sd_M3^2 + 2 * log(sd_M3),
    lwr_M3 = mean_M3 - qnorm(0.975) * sd_M3,
    upr_M3 = mean_M3 + qnorm(0.975) * sd_M3,
    is_M3 = (upr_M3 - lwr_M3) +
      (2 / 0.05) * pmax(lwr_M3 - log(count + 1), 0) +
      (2 / 0.05) * pmax(log(count + 1) - upr_M3, 0)
  )

# Summarise scores by month
cv_month_allmodels <- score_month %>%
  group_by(month) %>%
  summarise(
    RMSE_M0 = round(sqrt(mean(se_M0)), 1),
    MAE_M0  = round(mean(abs(count - mean_M0)), 1),
    DS_M0   = round(mean(ds_M0), 2),
    IS_M0   = round(mean(is_M0), 1),

    RMSE_M1 = round(sqrt(mean(se_M1)), 1),
    MAE_M1  = round(mean(abs(count - mean_M1)), 1),
    DS_M1   = round(mean(ds_M1), 2),
    IS_M1   = round(mean(is_M1), 1),

    RMSE_M2 = round(sqrt(mean(se_M2)), 1),
    MAE_M2  = round(mean(abs(count - mean_M2)), 1),
    DS_M2   = round(mean(ds_M2), 2),
    IS_M2   = round(mean(is_M2), 1),

    RMSE_M3 = round(sqrt(mean(se_M3)), 1),
    MAE_M3  = round(mean(abs(count - mean_M3_count)), 1),
    DS_M3   = round(mean(ds_M3), 2),
    IS_M3   = round(mean(is_M3), 1)
  )

print(cv_month_allmodels)
```

```r
#Model3

cv_table2 <- score_month %>%
  group_by(month) %>%
  summarise(
    RMSE = round(sqrt(mean(se_M3)), 1),
    DS   = round(mean(ds_M3), 2),
    .groups = "drop"
  ) %>%
  mutate(
    Month = month.abb[as.integer(month)]
  ) %>%
  select(Month, RMSE, DS)

print(cv_table2)


# 7. Exploratory summaries and plots

summary_stats <- data.frame(
  Variable = c("Daily Count", "Mean Temp (°C)", "Min Temp (°C)", "Max Temp (°C)"),
  Mean = round(c(mean(cycle_daily_df$count),
                 mean(cycle_daily_df$temp_mean),
                 mean(cycle_daily_df$temp_min),
                 mean(cycle_daily_df$temp_max)), 1),
  SD = round(c(sd(cycle_daily_df$count),
               sd(cycle_daily_df$temp_mean),
               sd(cycle_daily_df$temp_min),
               sd(cycle_daily_df$temp_max)), 1),
  Min = round(c(min(cycle_daily_df$count),
                min(cycle_daily_df$temp_mean),
                min(cycle_daily_df$temp_min),
                min(cycle_daily_df$temp_max)), 1),
  Median = round(c(median(cycle_daily_df$count),
                   median(cycle_daily_df$temp_mean),
                   median(cycle_daily_df$temp_min),
                   median(cycle_daily_df$temp_max)), 1),
  Max = round(c(max(cycle_daily_df$count),
                max(cycle_daily_df$temp_mean),
                max(cycle_daily_df$temp_min),
                max(cycle_daily_df$temp_max)), 1)
)

print(summary_stats)

#Covid Indicator
cycle_daily_df <- cycle_daily_df %>%
  mutate(
    is_covid = ifelse(year %in% 2020:2021, "COVID Period", "Post-COVID")
  )


#EDA Plots
```

```r
eda_time_series_plot <- ggplot(cycle_daily_df, aes(x = date, y = count)) +
  geom_line(alpha = 0.3, colour = "steelblue") +
  geom_smooth(colour = "darkred", se = FALSE) +
  labs(
    title = "Daily Cyclist Counts in Edinburgh, 2020-2025",
    x = "Date",
    y = "Daily Count"
  ) +
  theme_minimal()

eda_month_boxplot <- ggplot(cycle_daily_df, aes(x = month, y = count)) +
  geom_boxplot(fill = "blue", alpha = 0.6) +
  labs(
    title = "Count by Month",
    x = "Month",
    y = "Daily Count"
  ) +
  theme_minimal()

eda_dow_boxplot <- ggplot(cycle_daily_df, aes(x = dow, y = count)) +
  geom_boxplot(fill = "red", alpha = 0.6) +
  labs(
    title = "Count by Day of Week",
    x = "Day of Week",
    y = "Daily Count"
  ) +
  theme_minimal()

eda_temp_scatter <- ggplot(cycle_daily_df, aes(x = temp_mean, y = count)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "loess", colour = "red", se = FALSE) +
  labs(
    title = "Daily Cyclist Count vs Mean Temperature",
    x = "Mean Temperature (°C)",
    y = "Daily Count"
  ) +
  theme_minimal()

eda_covid_temp_plot <- ggplot(cycle_daily_df, aes(x = temp_mean, y = count, colour = is_covid)) +
  geom_point(alpha = 0.2) +
  geom_smooth(se = FALSE) +
  scale_colour_manual(values = c("COVID Period" = "darkred",
                                 "Post-COVID" = "steelblue")) +
  labs(
    title = "Cyclist Count vs Temperature by COVID Period",
    x = "Mean Temperature (°C)",
    y = "Daily Count",
    colour = ""
  ) +
  theme_minimal()

print(eda_time_series_plot)
print(eda_month_boxplot)
```

```r
print(eda_dow_boxplot)
print(eda_temp_scatter)
print(eda_covid_temp_plot)

# 8 Baseline model outputs

m0_coef_table <- broom::tidy(m0) %>%
  mutate(
    estimate = round(estimate, 3),
    std.error = round(std.error, 3),
    statistic = round(statistic, 2),
    p.value = round(p.value, 4)
  )

print(m0_coef_table)

m0_diag_data <- data.frame(
  fitted = fitted(m0),
  resid = resid(m0),
  std_resid = rstandard(m0)
)

print(m0_diag_data)

m0_resid_fitted_plot <- ggplot(m0_diag_data, aes(x = fitted, y = resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  labs(
    title = "M0: Residuals vs Fitted",
    x = "Fitted values",
    y = "Residuals"
  ) +
  theme_minimal()

print(m0_resid_fitted_plot)

m0_qq_plot <- ggplot(m0_diag_data, aes(sample = std_resid)) +
  stat_qq(alpha = 0.4) +
  stat_qq_line(colour = "red") +
  labs(
    title = "M0: Normal Q-Q Plot",
    x = "Theoretical Quantiles",
    y = "Standardised Residuals"
  ) +
  theme_minimal()

print(m0_qq_plot)

# 9. Final model outputs (M3)

m3_coef_table <- broom::tidy(m3) %>%
  mutate(
    estimate = round(estimate, 3),
```

```r
    std.error = round(std.error, 3),
    statistic = round(statistic, 2),
    p.value = round(p.value, 4)
  )

print(m3_coef_table)

m1_diag_data <- data.frame(
  fitted = fitted(m1),
  resid = resid(m1),
  std_resid = rstandard(m1)
)

m2_diag_data <- data.frame(
  fitted = fitted(m2),
  resid = resid(m2),
  std_resid = rstandard(m2)
)

m3_diag_data <- data.frame(
  fitted = fitted(m3),
  resid = resid(m3),
  std_resid = rstandard(m3)
)

m1_resid_fitted_plot <- ggplot(m1_diag_data, aes(x = fitted, y = resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  labs(title = "M1: Residuals vs Fitted", x = "Fitted values", y = "Residuals") +
  theme_minimal()

m1_qq_plot <- ggplot(m1_diag_data, aes(sample = std_resid)) +
  stat_qq(alpha = 0.4) +
  stat_qq_line(colour = "red") +
  labs(title = "M1: Normal Q-Q Plot",
       x = "Theoretical Quantiles", y = "Standardised Residuals") +
  theme_minimal()

m2_resid_fitted_plot <- ggplot(m2_diag_data, aes(x = fitted, y = resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  labs(title = "M2: Residuals vs Fitted", x = "Fitted values", y = "Residuals") +
  theme_minimal()

m2_qq_plot <- ggplot(m2_diag_data, aes(sample = std_resid)) +
  stat_qq(alpha = 0.4) +
  stat_qq_line(colour = "red") +
  labs(title = "M2: Normal Q-Q Plot",
       x = "Theoretical Quantiles", y = "Standardised Residuals") +
  theme_minimal()

m3_resid_fitted_plot <- ggplot(m3_diag_data, aes(x = fitted, y = resid)) +
  geom_point(alpha = 0.4) +
```

```r
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  labs(title = "M3: Residuals vs Fitted", x = "Fitted values", y = "Residuals") +
  theme_minimal()

m3_qq_plot <- ggplot(m3_diag_data, aes(sample = std_resid)) +
  stat_qq(alpha = 0.4) +
  stat_qq_line(colour = "red") +
  labs(title = "M3: Normal Q-Q Plot",
       x = "Theoretical Quantiles", y = "Standardised Residuals") +
  theme_minimal()

print(m1_resid_fitted_plot)
print(m1_qq_plot)
print(m2_resid_fitted_plot)
print(m2_qq_plot)
print(m3_resid_fitted_plot)
print(m3_qq_plot)

# 10. Table 3: trend coefficients from M1-M3

m1_daily_change <- coef(m1)["trend"]
m2_daily_change <- coef(m2)["trend"]

start_data <- cycle_daily_df %>%
  filter(date == as.Date("2020-01-01")) %>%
  slice(1)

start_pred_M3 <- exp(predict(m3, newdata = start_data)) - 1
m3_daily_change <- (start_pred_M3 + 1) * coef(m3)["trend"]

trend_coefs <- data.frame(
  Model = c("M1", "M2", "M3"),
  `Daily change (cyclists/day)` = round(c(m1_daily_change, m2_daily_change, m3_daily_change), 3),
  `Annual change (cyclists/year)` = round(c(m1_daily_change, m2_daily_change, m3_daily_change) * 365, 0
)

print(trend_coefs)

# 10b. Trend significance for M1-M3

trend_significance <- data.frame(
  Model = c("M1", "M2", "M3"),
  Estimate = c(
    summary(m1)$coefficients["trend", "Estimate"],
    summary(m2)$coefficients["trend", "Estimate"],
    summary(m3)$coefficients["trend", "Estimate"]
  ),
  Std_Error = c(
    summary(m1)$coefficients["trend", "Std. Error"],
    summary(m2)$coefficients["trend", "Std. Error"],
    summary(m3)$coefficients["trend", "Std. Error"]
  ),
  t_value = c(
```

```r
    summary(m1)$coefficients["trend", "t value"],
    summary(m2)$coefficients["trend", "t value"],
    summary(m3)$coefficients["trend", "t value"]
  ),
  p_value = c(
    summary(m1)$coefficients["trend", "Pr(>|t|)"],
    summary(m2)$coefficients["trend", "Pr(>|t|)"],
    summary(m3)$coefficients["trend", "Pr(>|t|)"]
  )
) %>%
  mutate(
    Estimate = round(Estimate, 4),
    Std_Error = round(Std_Error, 4),
    t_value = round(t_value, 2),
    p_value = round(p_value, 4)
  )

print(trend_significance)

# 10c. Annual growth as percentage of predicted count on 2020-01-01

start_data <- cycle_daily_df %>%
  filter(date == as.Date("2020-01-01")) %>%
  slice(1)

start_pred_M1 <- predict(m1, newdata = start_data)
start_pred_M2 <- predict(m2, newdata = start_data)
start_pred_M3 <- exp(predict(m3, newdata = start_data)) - 1

m1_daily_change <- coef(m1)["trend"]
m2_daily_change <- coef(m2)["trend"]

m3_trend_log <- coef(m3)["trend"]
m3_daily_change <- (start_pred_M3 + 1) * m3_trend_log

growth_rate_table <- data.frame(
  Model = c("M1", "M2", "M3"),
  Predicted_start_count = c(start_pred_M1, start_pred_M2, start_pred_M3),
  Daily_change = c(m1_daily_change, m2_daily_change, m3_daily_change)
) %>%
  mutate(
    Annual_change = Daily_change * 365,
    Growth_percent = 100 * Annual_change / Predicted_start_count,
    Predicted_start_count = round(Predicted_start_count, 1),
    Daily_change = round(Daily_change, 3),
    Annual_change = round(Annual_change, 0),
    Growth_percent = round(Growth_percent, 2)
  ) %>%
  select(Model, Predicted_start_count, Daily_change, Annual_change, Growth_percent)

print(growth_rate_table)

# 11. Section 5.1: compare temperature variables in M2
```

```r
temp_model_formulas <- list(
  temp_mean = count ~ weekend + trend + factor(month) + factor(dow) +
    temp_mean + I(temp_mean^2),
  temp_min = count ~ weekend + trend + factor(month) + factor(dow) +
    temp_min + I(temp_min^2),
  temp_max = count ~ weekend + trend + factor(month) + factor(dow) +
    temp_max + I(temp_max^2)
)

temp_cv_results <- bind_rows(lapply(names(temp_model_formulas), function(temp_name) {
  bind_rows(lapply(years, function(yr) {

    test_df  <- cycle_cv_df %>% filter(year == yr)
    train_df <- cycle_cv_df %>% filter(year != yr)

    fit <- lm(temp_model_formulas[[temp_name]], data = train_df)
    pred <- predict(fit, newdata = test_df, se.fit = TRUE)

    mu <- pred$fit
    sigma <- sqrt(summary(fit)$sigma^2 + pred$se.fit^2)
    y <- test_df$count
    scores <- calc_scores(y, mu, sigma)

    data.frame(
      temp_var = temp_name,
      year = yr,
      RMSE = scores$RMSE,
      DS = mean(scores$DS),
      IS = mean(scores$IS)
    )
  }))
}))

temp_compare_table <- temp_cv_results %>%
  group_by(temp_var) %>%
  summarise(
    RMSE = round(mean(RMSE), 1),
    DS   = round(mean(DS), 2),
    IS   = round(mean(IS), 1),
    .groups = "drop"
  )



# 12. Marginal temperature effects for chosen temperature variable

print(temp_compare_table)

m2_temp_min <- lm(
  count ~ weekend + trend + factor(month) + factor(dow) +
    temp_min + I(temp_min^2),
  data = cycle_daily_df
)
```

```r
beta1 <- coef(m2_temp_min)["temp_min"]
beta2 <- coef(m2_temp_min)["I(temp_min^2)"]

marginal_effect_5 <- beta1 + 2 * beta2 * 5
marginal_effect_15 <- beta1 + 2 * beta2 * 15

temp_effect_table <- data.frame(
  Temperature = c("5C", "15C"),
  `Marginal effect (cyclists per +1C)` = round(c(marginal_effect_5, marginal_effect_15), 1)
)

print(temp_effect_table)

# 13. COVID indicator extension check

cycle_daily_df <- cycle_daily_df %>%
  mutate(is_covid_binary = ifelse(year %in% 2020:2021, 1, 0))

start_data <- cycle_daily_df %>%
  filter(date == as.Date("2020-01-01")) %>%
  slice(1)

m3_covid <- lm(
  log(count + 1) ~ weekend + trend + factor(month) + factor(dow) +
    temp_mean + I(temp_mean^2) + is_covid_binary,
  data = cycle_daily_df
)

start_pred_M3 <- exp(predict(m3, newdata = start_data)) - 1
start_pred_M3_covid <- exp(predict(m3_covid, newdata = start_data)) - 1

m3_daily_change <- (start_pred_M3 + 1) * coef(m3)["trend"]
m3_covid_daily_change <- (start_pred_M3_covid + 1) * coef(m3_covid)["trend"]

trend_comparison <- data.frame(
  Model = c("M3", "M3 + COVID"),
  Daily_change = round(c(m3_daily_change, m3_covid_daily_change), 3),
  Annual_change = round(c(m3_daily_change, m3_covid_daily_change) * 365, 0)
)

print(trend_comparison)

# 14. 2035 extrapolation under fixed summer weekday conditions

future_dates <- data.frame(
  date = seq(as.Date("2020-01-01"), as.Date("2035-12-31"), by = "month")
) %>%
  mutate(
    year = year(date),
    month = factor(7, levels = 1:12, ordered = TRUE),   # July
    dow = factor("Wed", levels = c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"),
                 ordered = TRUE),
    weekend = 0,
```

```r
    temp_mean = mean(cycle_daily_df$temp_mean, na.rm = TRUE),
    trend = as.integer(date - as.Date("2020-01-01"))
  )

future_dates$pred_log <- predict(m3, newdata = future_dates)
future_dates$pred_count <- exp(future_dates$pred_log) - 1

extrapolation_plot <- ggplot(future_dates, aes(x = date, y = pred_count)) +
  geom_line(colour = "darkblue") +
  geom_hline(yintercept = 15000, linetype = "dashed", colour = "red") +
  labs(
    title = "Projected Cycling Demand to 2035 under Summer Weekday Conditions",
    x = "Date",
    y = "Predicted Daily Count"
  ) +
  theme_minimal()

print(extrapolation_plot)

threshold_15000 <- future_dates %>%
  filter(pred_count >= 15000) %>%
  slice(1) %>%
  select(date, pred_count)

print(threshold_15000)

# 15. Poorly fit period for final model

cycle_daily_df$m3_fitted_log <- fitted(m3)
cycle_daily_df$m3_fitted_count <- exp(cycle_daily_df$m3_fitted_log) - 1
cycle_daily_df$m3_resid_count <- cycle_daily_df$count - cycle_daily_df$m3_fitted_count

resid_time_plot <- ggplot(cycle_daily_df, aes(x = date, y = m3_resid_count, colour = factor(year))) +
  geom_line() +
  labs(
    title = "Final Model Residuals over Time",
    x = "Date",
    y = "Residual (count scale)",
    colour = "Year"
  ) +
  theme_minimal()

print(resid_time_plot)

poor_period_table <- cycle_daily_df %>%
  mutate(year_month = format(date, "%Y-%m")) %>%
  group_by(year_month) %>%
  summarise(
    mean_residual = mean(m3_resid_count),
    RMSE = sqrt(mean(m3_resid_count^2)),
    .groups = "drop"
  ) %>%
  arrange(desc(RMSE))
```

```r
print(head(poor_period_table, 10))

# 16. Original plot: monthly prediction bias over time

bias_plot_data <- cycle_daily_df %>%
  mutate(year_month = format(date, "%Y-%m")) %>%
  group_by(year_month) %>%
  summarise(
    mean_residual = mean(m3_resid_count),
    .groups = "drop"
  )

original_bias_plot <- ggplot(bias_plot_data, aes(x = year_month, y = mean_residual, group = 1)) +
  geom_line(colour = "purple") +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red") +
  labs(
    title = "Monthly Mean Prediction Bias of Final Model",
    x = "Year-Month",
    y = "Mean Residual"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5))

print(original_bias_plot)
```